

SELVEDGE

Product Requirements Document
Computational Upcycling · v1.0 Personal Tool

Version	1.0 — Personal Tool
Date	May 2026
Scope	Alter Mode (all garments) + Remake Mode (Skirts & Trousers)
Stack	Python · PyTorch · OpenCV · Shapely · Streamlit · Anthropic API
Fabric constraint	Woven stable fabrics only (no stretch/knit)
Author	Personal project — ML/AI for Engineering

What is Selvedge?

Selvedge is a personal computational tool that uses computer vision and machine learning to help you upcycle thrifted garments. Photograph a piece of clothing, tell Selvedge what you want to do with it — alter it to fit your body, or deconstruct it into something entirely new — and it generates precise, laser-cuttable pattern pieces optimized to your measurements and available fabric yield.

1. Problem & Motivation

Thrift shopping offers access to beautiful, unique fabrics at low cost — but garments rarely fit off the rack. The standard options are to leave them as-is, pay for tailoring, or attempt alterations manually. None of these options leverage the full creative potential of the fabric.

Upcycling — deconstructing a donor garment and remaking it into something new — solves this, but introduces a different problem: you must commit to seam-ripping before you know whether the fabric yield is sufficient for your intended design. This creates a high-stakes, irreversible decision made with incomplete information.

Selvage removes that uncertainty. It digitizes the donor garment from photographs, estimates what is possible before any fabric is cut, and generates fitted pattern pieces for the chosen outcome — closing the loop between thrift store find and finished garment.

1.1 Personal Context

This project extends prior work in computational sewing: a parametric Autodesk Fusion 360 pipeline that generated clothing patterns from user measurements and output laser-cuttable files. Selvage replaces the manual CAD workflow with an ML-powered pipeline, incorporating studies from a Master's in AI/ML for Engineering. The primary ML contributions are:

- Computer vision pipeline for garment digitization from flat photographs
- Garment classification model (fine-tuned on DeepFashion / iMaterialist)
- Fabric yield estimation with a self-improving regression model
- Cutting layout optimizer using No-Fit Polygon nesting + learned heuristics

2. Goals & Non-Goals

2.1 Goals

V1 Personal Tool

- Digitize a garment from 2 flat photographs (front + back) using CV
- Store and manage a personal body measurement profile
- Alter Mode: produce fit-delta overlay + plain-language adjustment instructions
- Remake Mode: estimate fabric yield pre-seam-ripping and surface ranked garment options
- Generate laser-cuttable SVG/DXF pattern pieces sized to body measurements
- Optimize cutting layout for available donor fabric panels
- Run entirely locally via a Streamlit web app — no cloud dependency
- Persist garment history and fit feedback to build personal training data over time

V2 Targets (out of scope for V1)

- Tops category (shirts, blouses, bodices) — priority after bottoms pipeline validated
- Mobile PWA with direct phone camera integration
- Knit and stretch fabrics (requires separate ease and grain line logic)
- Research packaging and publication
- Multi-user support, hosted inference, authentication

2.2 Non-Goals

- Fusion 360 or any proprietary CAD dependency
- 3D garment simulation or virtual try-on
- E-commerce, pattern marketplace, or social sharing features
- Support for woven fabrics with significant stretch or bias-cut designs in V1
- Automated purchasing recommendations

3. Target User & Use Cases

3.1 Primary User

Profile	Intermediate sewist — comfortable with commercial patterns, has done basic alterations
Experience	Owns a sewing machine, can interpret a pattern piece, no CAD background required
Motivation	Thrifts regularly, cares about sustainable fashion, frustrated by fit and wants to upcycle
Technical comfort	Comfortable with a laptop and browser-based tools; access to a laser cutter is a plus but not required

3.2 Core Use Cases

Use Case 1 — Alter a thrifted garment

1. User photographs garment laid flat, front and back
2. CV pipeline classifies garment type and extracts key dimensions
3. System compares garment dimensions to stored body profile
4. User receives: fit delta overlay on garment photo + plain-language caption (e.g. "Take in side seams 1.5 inches from hip to waist")
5. Optional: export modified panel as SVG pattern piece for precision alteration

Use Case 2 — Remake a thrifted garment into something new

6. User photographs donor garment laid flat, front and back
7. System classifies garment, estimates fabric yield (before seam-ripping)
8. System presents ranked garment categories feasible within yield (e.g. Skirt ✓, Trousers ✓, Dress ✗)
9. User selects category → selects style (e.g. Skirt → A-line)
10. System checks fabric sufficiency; if short, LLM suggests adaptive modifications or second donor piece
11. System generates optimized cutting layout + pattern pieces sized to body profile
12. User receives: SVG/DXF pattern pieces + assembly instructions + cutting layout diagram
13. After making: user logs fit feedback for model improvement

4. System Architecture

Selvedge is a local Python application. All ML inference runs on the user's machine. No data leaves the device. The Anthropic API is the only external network call (LLM creative suggestion layer).

4.1 High-Level Flow

 Photo Input User uploads front + back flat garment photos via Streamlit
 CV Pipeline Garment classification → silhouette extraction → dimension measurement → seam detection (Remake only)
 Fit Engine Rule-based ease (Aldrich/Armstrong) + body profile delta calculation
 ML Layer Yield estimation model · Layout optimization model · Fit feedback model
 Pattern Engine Shapely geometry → parametric pattern pieces per style → SVG/DXF output
 LLM Layer Claude API — adaptive design suggestions when fabric is constrained
 Interface Streamlit local web app — photo upload, mode toggle, inline visual outputs

4.2 CV Pipeline Detail

The computer vision pipeline operates in layers of increasing complexity, with different layers activated depending on mode:

Layer 1	Silhouette + key dimensions — object segmentation, perspective correction, key measurement extraction (both modes)
Layer 2	Garment classification — CNN fine-tuned on DeepFashion/iMaterialist; outputs garment type (both modes)
Layer 3	Seam detection — identifies panel boundaries on garment surface for panel geometry inference (Remake mode only)

Layer 4	Panel geometry reconstruction — full 2D panel shapes including curves, enabled after seam-ripping confirmation (Remake mode only)
----------------	---

4.3 ML Components

Garment Classification Model

- Architecture: Fine-tuned ResNet or EfficientNet on DeepFashion2 / iMaterialist Fashion 2019
- Input: Flat garment photo (front), normalized
- Output: Garment category (skirt, trouser, shirt, jacket, dress, other) + confidence
- Evaluation metric: Top-1 accuracy on held-out thrifted garment photos

Fabric Yield Estimation Model

- V1 baseline: Lookup table (garment type × approximate size) + geometric correction from silhouette area
- V2 ML layer: Regression model trained on personal dataset — (garment photo, garment type, silhouette dimensions) → fabric yield in m²
- Training data: Self-generated — every seam-ripped garment contributes one labeled example (photo → measured yield)
- Model improves organically through use; meaningful signal after ~20 labeled examples

Cutting Layout Optimizer

- Baseline: No-Fit Polygon (NFP) nesting — places pattern pieces into irregular donor panel shapes
- Implementation: Shapely (Python) for polygon operations; NFP computed via Minkowski difference
- ML enhancement: Learned piece ordering and rotation heuristics — model trained on (donor panel shapes, pattern pieces) → nesting sequence that maximizes fabric utilization
- Research framing: Learned layout optimization for upcycling-constrained garment pattern nesting

Fit Feedback Model

- Input: (body measurements, garment measurements, garment type, ease rules applied) → fit quality per zone
- Training signal: User-logged feedback after wearing each made garment
- Function: Personalizes ease values over time beyond standard Aldrich/Armstrong tables
- Activates meaningfully after ~10–15 fit feedback entries

5. Feature Specification

5.1 Onboarding — Body Profile

- 6 core measurements collected at first launch: bust/chest, waist, hips, inseam, height, shoulder width
- Progressive disclosure: additional measurements (upper arm, rise, back length, thigh, wrist, neck) prompted in-context when a garment requires them
- Profile stored as local JSON; updated incrementally over time
- Target onboarding time: under 3 minutes
- Each measurement accompanied by a simple diagram showing where to measure

5.2 Alter Mode

Input	2 flat garment photos (front + back) + stored body profile
CV output	Garment type classification + key dimension extraction (chest, waist, hip, length, shoulder)
Fit calculation	Rule-based ease delta per garment type using Aldrich/Armstrong tables
Primary output	Fit overlay image — garment photo annotated with adjustment zones and measurements
Caption	Plain-language instruction per adjustment zone (e.g. "Take in 1.5" at side seam, hip to waist")
Optional export	Modified pattern panel as SVG, sized to body measurements
Conflict handling	Flags adjustments that exceed safe seam allowance or require structural changes

5.3 Remake Mode

Input	2 flat donor garment photos (front + back) + stored body profile + desired category/style
Flow	Garment-first: inventory → category selection → style selection → pattern generation
Yield estimation	Lookup table + geometric correction (V1); regression model (V2)
Category ranking	Garment categories ranked by feasibility against yield estimate and body measurements
Conflict handling	Soft warning if yield insufficient; LLM suggests adaptive design (fewer tiers, shorter length, contrast panel from second donor piece)
Pattern output	SVG/DXF pattern pieces, grain lines marked, seam allowance included (default 1.5cm)

Layout output	Cutting layout diagram — optimal placement of pattern pieces within donor panel shapes
Assembly output	Plain-text sewing order with key notes per seam

5.4 Garment Taxonomy — V1 Scope

Category	Styles	Notes
Skirt	A-line, Wrap, Tiered/Gathered	<i>Simplest geometry; good pipeline validation target</i>
Trouser	Straight leg, Wide leg, Shorts	<i>Introduces rise and inseam calculations</i>
Tops (V2)	TBD — shirts, blouses, bodices	<i>Adds armhole, collar, sleeve complexity</i>

5.5 LLM Creative Layer

- Powered by Claude API (claude-sonnet-4-20250514)
- Activated when: fabric yield is insufficient for chosen style, or user requests creative suggestions
- Input context: donor panel dimensions, yield estimate, target garment type and style, body measurements
- Output: 2–3 adaptive design suggestions in plain language with fabric math shown
- Example: "You have 1.2m² — enough for a 2-tier gathered skirt (needs 1.1m²) but not 3-tier (needs 1.6m²). Alternatively, pair with a second donor piece for contrast tiers."

6. Data Model & Persistence

All data stored locally on disk. No database required for V1. Structure designed to be incrementally populated — the tool is useful from day one and gets smarter over time.

6.1 File Structure

```
selvedge/
  body_profile.json
  /garments/
    /YYYY-MM-DD-{garment-name}/
      front.jpg · back.jpg
      cv_output.json      ← classification, dimensions, yield estimate
      remake_output/
        pattern_pieces.svg
        cutting_layout.svg
        assembly_notes.txt
        fit_feedback.json ← filled in after wearing
```

6.2 body_profile.json Schema

core_measurements	bust_cm, waist_cm, hip_cm, inseam_cm, height_cm, shoulder_width_cm
extended_measurements	upper_arm_cm, wrist_cm, thigh_cm, rise_cm, back_length_cm, neck_cm (added progressively)
fit_preferences	default_ease_preference: fitted relaxed oversized
metadata	created_at, last_updated, measurement_count

6.3 fit_feedback.json Schema

garment_type	Category and style made
date_worn	ISO date string
zone_feedback	Object: { shoulders, chest, waist, hip, length } each with value: perfect tight_{n}cm loose_{n}cm
overall_rating	1–5 integer
notes	Free text field for qualitative notes

7. Technical Stack

Layer	Technology	Purpose
Interface	Streamlit	Local web app — photo upload, mode selection, output display
CV & Image Processing	OpenCV + Pillow	Perspective correction, segmentation preprocessing
ML Framework	PyTorch	Garment classification, yield regression, layout optimizer
Geometry Engine	Shapely	Pattern piece generation, NFP nesting, SVG polygon ops
Pattern Output	svgwrite + ezdxf	SVG for display, DXF for laser cutter
LLM Layer	Anthropic Python SDK	Claude API for adaptive design suggestions
Data Storage	JSON / filesystem	Body profile, garment history, fit feedback
Training Datasets	DeepFashion2, iMaterialist	Garment classification fine-tuning base

8. Development Milestones

Phase	Milestone	Key Deliverable
M1	Body profile onboarding + Streamlit shell	body_profile.json populated; app runs locally
M2	CV pipeline — garment classification	Classify garment type from flat photo with >85% accuracy on personal test set
M3	CV pipeline — dimension extraction	Extract key measurements from flat photo within 1cm of tape measure ground truth
M4	Alter Mode — end to end	Fit overlay + caption output for a real thrifted garment
M5	Remake Mode — yield estimation (baseline)	Lookup table + geometric correction producing reasonable yield estimates
M6	Pattern engine — Skirts	A-line, Wrap, Tiered SVG pattern pieces generated for body measurements
M7	Pattern engine — Trousers	Straight leg, Wide leg, Shorts SVG pattern pieces generated
M8	Cutting layout optimizer (NFP baseline)	Pattern pieces nested into donor panel shapes with fabric utilization metric
M9	LLM creative layer	Claude API suggestions firing correctly for constrained-yield scenarios
M10	Fit feedback loop	Feedback JSON written after wearing; data accumulating for future model training
M11	Yield regression model (V2 ML)	Personal dataset of ~20 seam-ripped garments; regression model outperforms lookup table
M12	Layout optimization ML layer	Learned nesting heuristics trained on accumulated layout history

9. Risks & Mitigations

Risk	Impact	Mitigation
CV accuracy on diverse fabric textures and patterns	High	Collect a diverse personal test set early; augment training data with texture overlays; fall back to manual measurement input if confidence is low
Yield estimation error leading to fabric shortfall mid-project	High	Always add a 10% buffer to yield estimates; surface uncertainty range to user (e.g. "1.2–1.5m ² "); prompt user to measure one panel manually as a sanity check
NFP nesting complexity on highly irregular donor panels	Medium	Implement rectangular approximation as a fast fallback; NFP is incremental so partial layouts are still useful
Seam detection unreliable on busy prints or dark fabrics	Medium	Seam detection is Remake-only and post-seam-rip; user can manually confirm or override panel boundaries in the UI
Pattern ease rules don't match personal fit preferences	Medium	Fit feedback loop corrects this over time; add manual ease adjustment sliders in Streamlit for immediate override
Insufficient personal training data for ML models	Low	Rule-based baselines are fully functional without ML; ML layers are progressive enhancements, not V1 requirements

10. Future Directions

10.1 V2 — Tops

Tops introduce structural complexity absent in bottoms: armhole curves, collar and neckline geometry, sleeve insertion, and dart placement. The pattern engine will require new parametric rules per style. Recommended starting styles: boxy crop top (fewest seams), button-front shirt, sleeveless bodice.

10.2 Mobile PWA

The natural evolution once the desktop pipeline is stable. A progressive web app served on local WiFi allows the phone camera to feed directly into the CV pipeline without a transfer step — removing the largest friction point in the current workflow. Flask + a simple mobile-optimized frontend can replace Streamlit for this use case.

10.3 Research Packaging

The system's three ML contributions — garment classification from flat photographs, fabric yield estimation with self-generated training data, and learned cutting layout optimization — each represent tractable research contributions. The layout optimization framing (learned heuristics for upcycling-constrained 2D bin packing) is the strongest candidate for a conference submission given the novelty of the application domain.

10.4 Multi-Donor Composition

A natural extension of the LLM suggestion layer: reasoning over two or more donor garments simultaneously to compose a new piece from mixed fabrics. "You have a linen blazer and a cotton shirt — here is a contrast-panel skirt that uses both." Requires minimal architectural change beyond expanding the yield input context.